The Minions turn-lattice player

System description · champion net: gen 660 · weights in `weights/minions/lattice-gen660`

> **Overview.** The player searches in two tiers. Tier 1 is AlphaZero-style MCTS over *complete turns*, using progressive widening. Tier 2 is a second MCTS over the *lattice of partial turns*: sets of commitments ordered by inclusion. Tier 2 proposes the turns that tier 1 compares. Two small networks drive both tiers:
>
> - a position trunk with a value head, 1.40M parameters;
> - a partial-turn head, 1.03M parameters.
>
> The nets train only on search targets from self-play, never on game results. The champion net plays every self-play game, and a candidate replaces it only by winning a gate match.

1. The problem: one turn is a combinatorial action

Minions is a two-player game on a 49-cell hex board, stored as a 7×7 grid. A player wins by killing the enemy necromancer, or on net worth at the 30-turn cap. Each turn has three parts:

- attacks, which decide the fate of each enemy unit;
- movement, which gives each friendly unit a destination;
- spawns, which buy units onto spawn cells with money.

The legal turns are the product of these choices, subject to a realizability constraint: some legal order of moves and attacks must achieve the declared outcome. That leaves far too many turns to enumerate, and their legality is expensive to check. The design gets around both problems by searching over *declarations* instead of move sequences. One side starts with extra money, the handicap, to offset the first-move advantage.

2. The turn lattice

A turn is a set of **commitments**:

- each enemy unit gets a fate (stays, killed, or unsummoned);
- each friendly unit gets a destination;
- each spawn cell gets a unit or nothing.

A **lattice node** is a partial set of commitments that can still be completed to a legal turn. Nodes are ordered by inclusion. The source is the empty declaration, the sinks are complete turns, and every edge adds one commitment. There is no fixed variable order: a node's children are every feasible (cell, choice) pair for every undecided variable of the current phase.

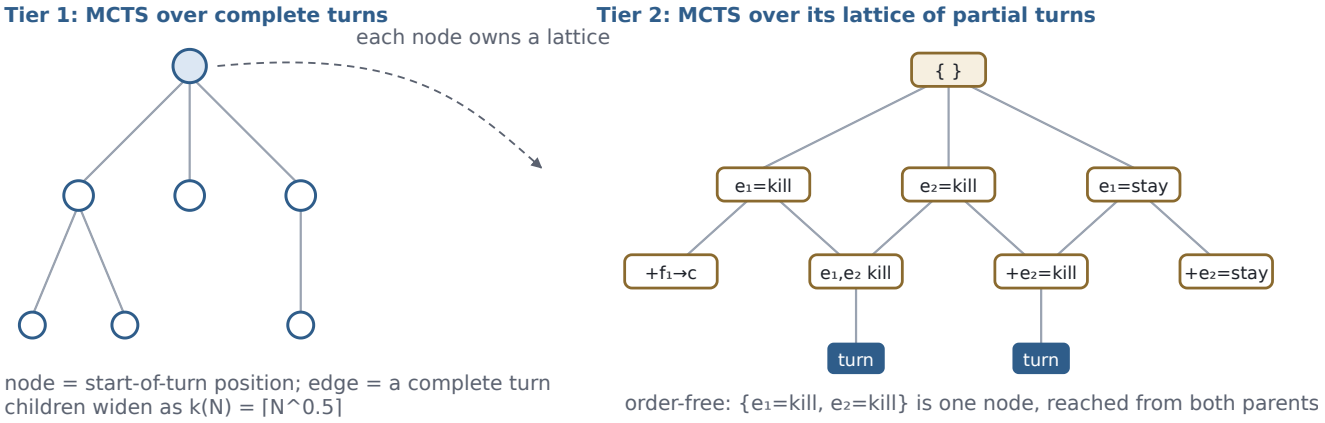


Figure 1. Two tiers. A tier-1 widening is one tier-2 descent from the lattice source to a sink, i.e. a complete turn. Lattice nodes are partial declarations ordered by inclusion, so different commitment orders reach the same node.

- **Phases.** Fates come first, until every enemy is fated. Then moves, until every friendly unit has a destination. The attack part is then realized. Spawns run last, on the real spawn-phase board, until none remain available. A spawn cell whose only remaining option is "nothing" is committed automatically.
- **Necromancer.** Its fate is never a choice. If the enemy necromancer can be killed this turn, the tier-1 node is terminal with that score. A complete turn after which the opponent can kill our necromancer is scored as a loss on the spot.
- **Keys and inclusion.** A node is a `mask`, one bit per variable, plus 6 bits per variable for the choice. Node X includes node P iff `P.mask & !X.mask == 0` and `(X.packed ^ P.packed) & field(P.mask) == 0`. That is a handful of word operations.
- **Feasibility.** Every offered child carries a *witness*, a concrete completion that proves it realizable. The checks are layered from cheap to expensive:

  1. cheap filters;
  2. a fast realizer;
  3. a Hall-type assignment bound;
  4. an exact search.

  Witnesses are checked lazily: a child's check runs when the child is first expanded. Each lattice keeps a pool of witnesses and a list of *nogoods*, keys proven infeasible, which prunes the nodes that include them. When a widening exceeds 1000 expansions, an exact SAT encoding of the attack phase takes over (CaDiCaL). It either returns a witness or refutes the node.
- **Repeated positions.** Friendly units may move in cycles, so distinct declarations can reach the same position. Tier 1 merges them by position: a turn that reaches a position an existing child already holds is a repeat, not a new child.

3. Tier 2: PUCT over the lattice

- **Selection.** From the source, take the child with the highest PUCT score ($c = 1.5$), expanding along the way, until a sink. Every descent is a whole source-to-turn path. Priors are a softmax within each cell, so each cell's choices sum to 1. A single PUCT comparison over every (cell, choice) pair therefore picks the cell and its choice at once.
- **Expansion** is one tier-2 net call. It returns, for each undecided cell:

    - prior logits over the cell's choices;
    - one value per child;
    - a value for the node itself.

  The expansion counts as direct visits at those values.
- **Sinks** are valued by the tier-1 value of the resulting position, flipped to the mover, or by their terminal score.
- **Lazy sublattice propagation.** A visit to a node counts as a visit to every node it includes. There is no path backup. The search keeps a list of direct visits and computes $N(P) = \Sigma_{X \supseteq P}\, n_X$ and $W(P)$ with one scan of that list per selection step, which credits each child of P at the same time.
- **Training targets come from complete turns only.** The lattice's own predicted values only steer the search. Every training value comes from sink returns: complete turns valued by tier 1.

4. Tier 1: AlphaZero over turns with progressive widening

- A tier-1 node is a start-of-turn position, and its children are the complete turns found so far. A visit **widens** while the node has fewer than $k(N) = \lceil N^{0.5} \rceil$ children. A widening is one tier-2 descent in that node's lattice. A repeat is retried up to 20 times, after which the node stops widening.
- PUCT with $c = 1.5$ and mean backup, with the side flipped at every edge.
- **The tier-1 prior is near-uniform:** softmax(child value / 100). Tier 1 ranks the turns well, but progressive widening only ever gives it $\sqrt{N}$ of them, all proposed by tier 2, so tier 2's quality bounds the search.
- **Visits in flight.** Each search keeps several visits running at once, with virtual loss along the path, so the net is called in batches. To keep this equivalent to a serial search:
    - a visit waits while its node has a widening pending and fewer than $k(N)$ children;
    - the root, which has no virtual loss, counts the other in-flight visits in its widening threshold.

5. Networks

net	architecture	input	output
tier-1 trunk + value	`pos:96x8` : 96 channels, 8 residual blocks, 1.40M params	the full-turn position only: unit and graveyard planes; money, reinforcement and turns-left scalars; a constant plane	value, plus a 96×7×7 trunk image (fp16), cached per position
tier-2 head	`96x6` , 1.03M params	the trunk image plus the *lattice view*: the original position; enemy fates; decided friends at their origin; final friendly units at their final cell; a from-spawn bit; projected scalars	46 slots per cell (3 fates, 37 move destinations, 6 spawns) as prior logits and child values; node value; position value; energy

- **One value function.** The exported tier-1 value is the tier-2 head's node value at the empty declaration. Tier 1 and tier 2 therefore cannot disagree about a turn start.
- **Partial turns never enter the trunk.** The trunk sees full-turn positions; decisions reach tier 2 only. Tier 1 trains on turn records and tier 2 on lattice records.

- An **energy head** (a softmax over a node's tier-1 children) is trained with weight 0.2. The search does not use it.
- **Runtime.** Two ONNX graphs run under TensorRT; the CUDA execution provider returns wrong values for some shapes. A single runner merges requests from all games into large batches. Tier-2 batches gather cached trunk images instead of recomputing the trunk.

6. Self-play

setting	value
self-play net	the champion
concurrency	16 workers × 16 games, 8 tier-1 visits in flight per search, games started over a 60 s ramp; ~7 GB RSS
starting position	every game from the initial position
handicap	uniform per game on −5..10 dollars, so the value head learns what money is worth
tier-1 search	800 visits per turn; prior softmax(value/100); c 1.5; widening $\lceil N^{0.5} \rceil$
turn choice	anchored (below): temperature 1, Dirichlet α 0.3, weight 0.25
time split per generation	400 s of self-play, then up to 190 s for the tier-2 phase

Anchored turn choice

1. Run the tier-1 search, then pick a turn from the root visits with temperature and Dirichlet noise.
2. Take a random lattice path to that turn and pick one node on it as the *anchor*.
3. Run a from-scratch tier-2 search rooted at the anchor, in a fresh lattice. Its record is a tier-2 training target.
4. Draw the played turn from the complete turns that search reached, by descents, with temperature and Dirichlet noise.

The other nodes on the path join the tier-2 phase queue with probability 0.15. Games can thus leave the top tier-1 turns for turns that share a reasonable partial declaration with them; this noise is the only source of opening variety.

Records

- **Tier-1 value targets:** the root of each searched turn, plus every tier-1 node with at least 16 visits, at the mean of its backed-up tier-1 values. That is about 26 records per searched turn.
- **Tier-2 targets** come from rooted searches: the anchors plus the tier-2 phase. Each search runs 128 descents, visiting every kid once first. A kid's value target is the best complete turn reached below it. The prior target is a per-cell softmax of those values at τ = 0.02.
- **Energy groups:** each turn start with its tier-1 children's declarations and visits.
- Game results are never trained on.

a typical generation	count
games / searched turns in 400 s	~270 / ~1,000
tier-1 records	~27,000
tier-2 records: anchors / tier-2 phase	~1,000 / ~3,000
tier-2 phase throughput	~56 searches/s, ~55k expansions/s

7. Training and gating

- **Learner** (`python/minions_lattice_learner.py`). Joint training:
  - tier-2 prior cross-entropy;
  - child-value and node-value BCE;
  - tier-1 value BCE, on 256 tier-1 records per step;
  - energy loss at weight 0.2.

  The tier-2 losses also train the trunk. AdamW at learning rate 10^{-3}, one epoch over a window of the last 8 generations. About 130 steps and 60 s per generation.
- **Cycle** (`data/models/minions/lattice537/cycle.sh`):
  - Net G−1 trains while generation G self-plays, about 8 minutes per generation.
  - Training continues a *chain*: each candidate starts from the previous one.
  - The champion plays the self-play games.
  - After a reset, no training runs until the window holds 4 generations of data.
- **Gate**, every 4 generations: 64 seat-swapped pairs (128 games) against the champion, at 100 tier-1 visits with 16 in flight. Each pair starts after 2 net-sampled opening turns at handicap 3, where equal nets are even. A candidate becomes champion at ≥ 0.55 (standard error ≈ 0.044).

8. Throughput and memory

- Self-play is bound by latency, not memory or GPU compute. Throughput tracks the total visits in flight, games × in-flight. About 2,048 in flight gives about 3 searched turns/s, whatever the split between games and in-flight visits.
- The single ONNX runner's per-call host cost dominates: 1–2 ms per call whatever the row count. The GPU is 16–40% busy.
- Memory is dominated by lattice nodes, about 25 per tier-1 visit. Nodes are kept compact:
  - a hashed key index;
  - witnesses stored as differences;
  - a packed node pool;
  - boxed rarely-used fields.

9. Open issues

1. **Max versus mean.** Rooted tier-2 value targets take the *best* complete turn below each kid, while every other target *averages* complete turns.
2. **The tier-2 phase sits idle for about 70% of its budget** (it finishes in ~55 of 190 s). Moving that time to self-play, or queuing more nodes, is free throughput.

3. **Handicap pricing.** Gates use a fixed handicap of 3. An even bid after the opening (`--even-bid`) becomes usable once the value head prices money reliably.
4. **The tier-1 prior is near-uniform,** and the energy head is trained but unused.

Appendix: running it

```
source scripts/ort-env.sh            # TensorRT runtime; required

# play / inspect in the UI (lattice panel on :7107)
./target/release/minions-engine serve --addr 127.0.0.1:7105 \
    --lattice-net vprior=100:weights/minions/lattice-gen660

# match two lattice nets (gate format: 2 sampled opening turns, handicap 3)
./target/release/minions-engine steps-match --a vprior=100:<dirA> \
    --b vprior=100:<dirB> --pairs 64 --n1 100 --threads 16

# the training cycle (detached); stop with: touch data/models/minions/lattice537/STOP
SP_CHAMP=1 MIN_WINDOW=4 T1_ARCH=pos:96x8 T2_ARCH=96x6 RESET_MIN=0 \
GEN_FLAGS="--workers 16 --games 16 --inflight 8 --ramp-secs 60 --anchor --from-start --
random-handicap" \
setsid nohup bash data/models/minions/lattice537/cycle.sh >>
data/models/minions/lattice537/cycle.log 2>&1 < /dev/null &
```

code	role
`crates/games/minions/src/lattice.rs`	lattice, tier-2 PUCT, lazy propagation, rooted searches
`tier1.rs`	tier-1 MCTS with progressive widening, in-flight handling
`latgen.rs`	self-play driver (`lattice-gen`), anchored choice, records
`lattice_ort.rs`, `lview.rs`, `sat.rs`	ONNX/TensorRT evaluator; lattice view; SAT check
`stepsp.rs`, `lattice_ui.rs`	matches and openings; the UI's lattice panel server
`python/minions_lattice_learner.py`	learner and ONNX export
`docs/minions-lattice.md`	the design document